# Lab Assessment Rubrics of Digital System Design (CPE-344)

| | |
|---|---|
| **Lab Number** | **02** |
| **Lab Title** | Introduction of Transmission Gate Primitives, Implementation of Half Subtractor and Full Subtractor using gate level modeling. |
| **Submitted by** | |
| **Name** | **Registration Number** |
| Sumaira Safeer | FA22-BCE-019 |

| Rubrics name & number | | Marks | |
|---|---|---|---|
| | | **In-Lab** | **Post-Lab** |
| **Engineering Knowledge** | **R2: Use of Engineering Knowledge and follow Experiment Procedures:** Ability to follow experimental procedures, control variables, and record proceduralsteps on lab report. | | |
| | **R3: Interpretation of Subject Knowledge:** Ability to interpret and explain mathematical and/or visual forms, includingequations, diagrams, graphics, figures, and tables. | | |
| **Problem Analysis** | **R6: Experimental Data Analysis:** Ability to interpret findings, compare them to values in the literature, identify weaknesses and limitations. | | |
| **Design** | **R7: Implementing Design Strategy:** Ability to execute a solution takinginto consideration design requirements and pertinent contextual elements. [Block Diagram/Flow chart/Circuit Diagram] | | |
| | **R8: Best Coding Standards:** Ability to follow the coding standards and programming practices. | | |
| **Modern ToolsUsage** | **R9: Understand Tools:** Ability to describe and explain the principles behind and applicability ofengineering tools. | | |
| | **R11: Tools Evaluation: Ability to simulate the experiment and them using hardware tools to verify The results.** | | |
| **Individualand Teamwork** | **R12: Individual Work Contributions:** Ability to carry out individual responsibilities. | | |
| | **R13: Management of Teamwork:** Ability to appreciate, understand and work with multidisciplinary teammembers. | | |

**Rubrics to follow**

| Rubrics | R2 | R3 | R5 | R6 | R7 | R8 | R9 | R11 | R12 | R13 |
|---|---|---|---|---|---|---|---|---|---|---|
| In-Lab | | | | | | | | | | |
| Post-Lab | | | | | | | | | | |
| | | | | | | | | | | |

## Lab No: 02

## Title:

Introduction of Transmission Gate Primitives, Implementation of Half Subtractor and Full Subtractor (by instantiation) using gate level modeling.

**Lab Objective:**

- To understand the working of Transmission Gate Primitives i-e; not, buf, bufif0, notif0, bufif1, notif1.
- To implement the half adder using gate level, dataflow and structural modeling.

**Task #01:**

Implement Half Subtractor using both gate level and dataflow model in Verilog.

**Introduction:**

A Half Subtractor is a combinational logic circuit that performs subtraction of two binary bits. It has two inputs (A, B) and two outputs: Difference (D) and Borrow (B_out).

**Truth Table:**

| A | B | Difference (D) | Borrow (B_out) |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 |

**Boolean Expressions:**

- Difference, **D = A ⊕ B**
- Borrow, **B_out = A’B**

**Verilog Code (Gate-Level Modeling):**

```
module half_subtractor_gate(A, B, D, Bout);
 input A, B;
 output D, Bout;
 xor (D, A, B);
 and (Bout, ~A, B);
endmodule
```

**Verilog Code (Dataflow Modeling):**

```
module half_subtractor_dataflow(A, B, D, Bout);
```

```
  input A, B;
  output D, Bout;
  assign D = A ^ B;
  assign Bout = (~A) & B;
endmodule
```

**Verilog Test Bench:**

```
module tb_half_subtractor;
  reg A, B;
  wire D, Bout;

  half_subtractor_dataflow uut (A, B, D, Bout);

  initial begin
    $monitor("A=%b B=%b | D=%b Bout=%b", A, B, D, Bout);
    A=0; B=0; #10;
    A=0; B=1; #10;
    A=1; B=0; #10;
    A=1; B=1; #10;
  end
endmodule
```

**Waveform:**

Following waveforms are verifying our truth table for the AND GATE logic.

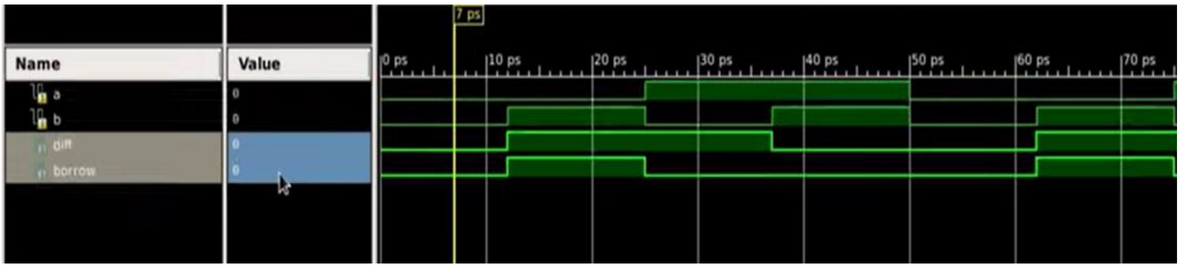



Figure 1: When both inputs are zero(0), our output is also zero(0)

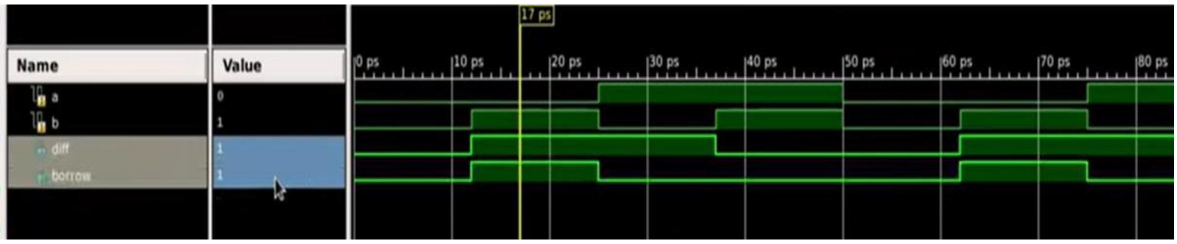



Figure 2: When our first input is zero (0) but second is one (1), but still our output remains zero (0)

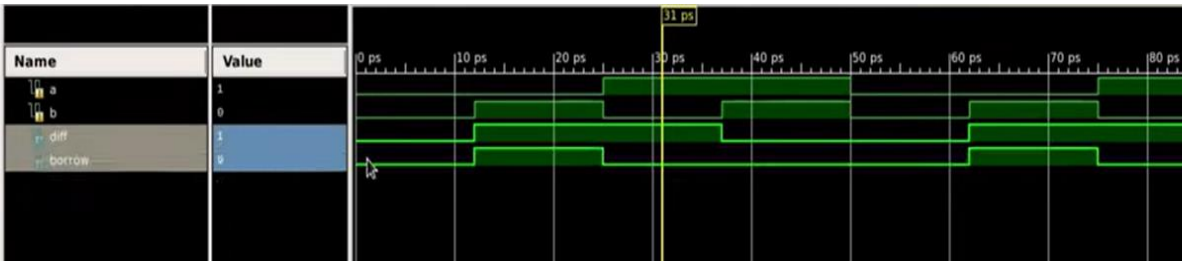



Figure 3: When our first input is one (1) but second is zero (0), but still output remains zero (0)

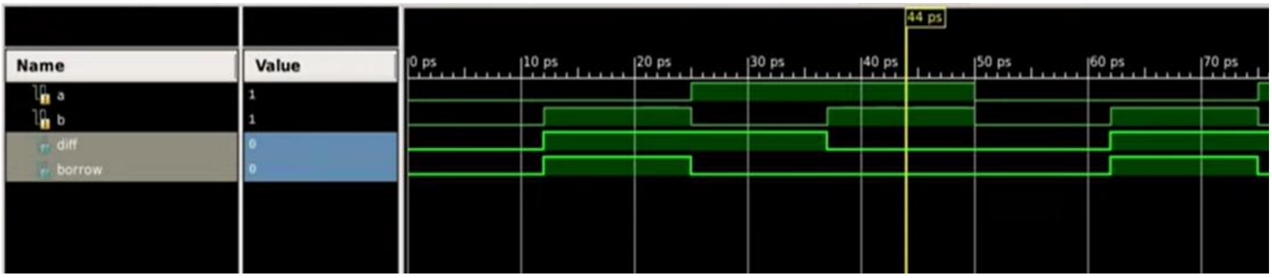



Figure 4 : When both inputs are one (1), output will be one (1)

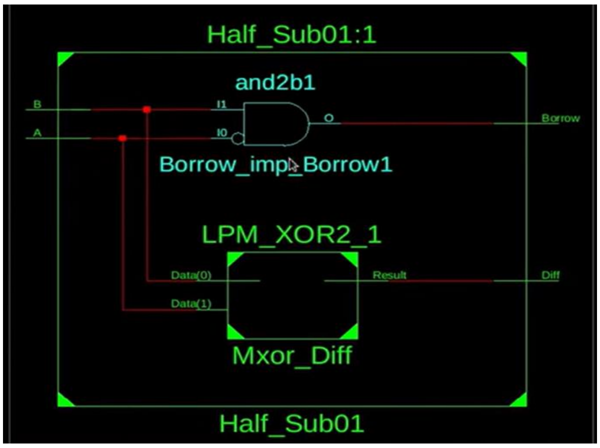
**Wave Description:**

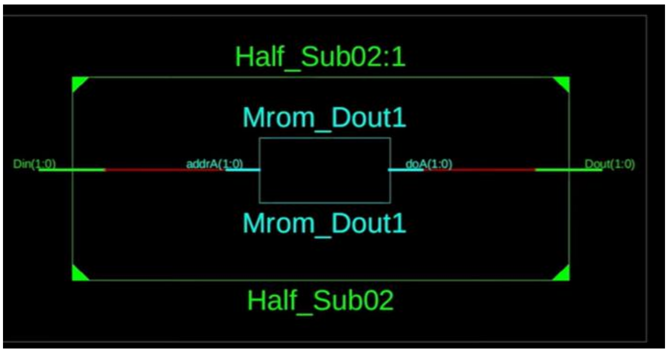
The waveform shows that:

- When A=0, B=1, both **Difference = 1** and **Borrow = 1**.
- When A=1, B=0, **Difference = 1** and **Borrow = 0**.

**RTL Diagram:**

The RTL diagram generated by Xilinx shows a direct **AND gate symbol** with two inputs and one output, confirming correct synthesis.






**Task #02:**

Implementation of Full Subtractor using two half Subtractors.

**Introduction:**

A Full Subtractor subtracts three binary bits — A (minuend), B (subtrahend), and Bin (borrow in) — producing a Difference (D) and a Borrow out (Bout).

**Truth Table:**

| A | B | Bin | D | Bout |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

**Boolean Expressions:**

- Difference, **D = A ⊕ B ⊕ Bin**
- Borrow, **Bout = (~A & B) | (B & Bin) | (~A & Bin)**

**Verilog Code (Using Instantiation):**

```
module half_subtractor(A, B, D, Bout);
 input A, B;
 output D, Bout;
 assign D = A ^ B;
 assign Bout = (~A) & B;
endmodule
```

```
module full_subtractor(A, B, Bin, D, Bout);
 input A, B, Bin;
 output D, Bout;
 wire D1, B1, B2;
 half_subtractor HS1(A, B, D1, B1);
 half_subtractor HS2(D1, Bin, D, B2);
 assign Bout = B1 | B2;
endmodule
```

**VerilogTest Bench:**

```
module tb_full_subtractor;
 reg A, B, Bin;
 wire D, Bout;
 full_subtractor uut (A, B, Bin, D, Bout);
 initial begin
  $monitor("A=%b B=%b Bin=%b | D=%b Bout=%b", A, B, Bin, D, Bout);
  A=0; B=0; Bin=0; #10;
  A=0; B=1; Bin=0; #10;
  A=1; B=0; Bin=1; #10;
  A=1; B=1; Bin=0; #10;
  A=1; B=1; Bin=1; #10;
 end
endmodule
```

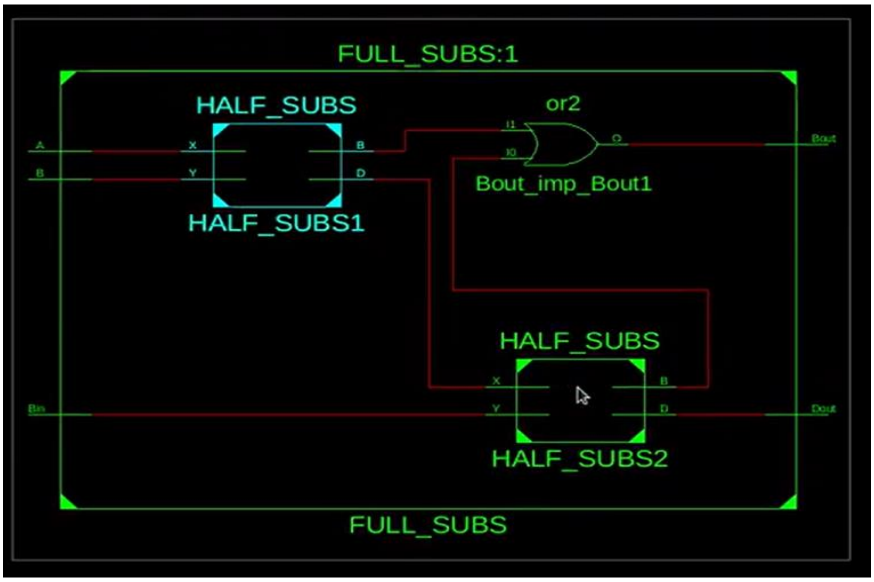
**Waveform:**

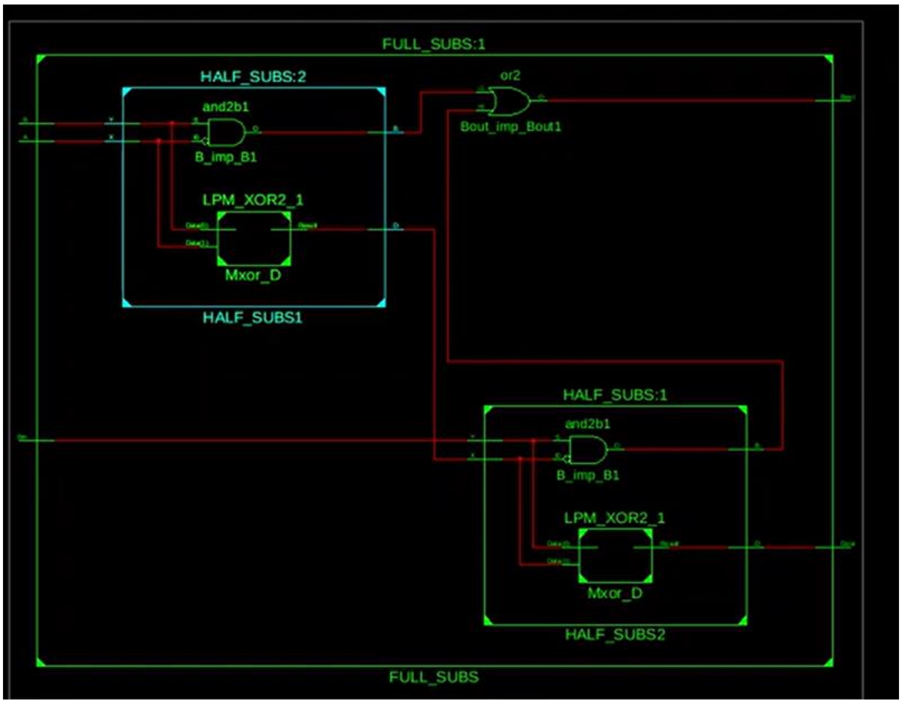

**Waveform Description:**

The waveform confirms correct subtraction logic:

- **Difference** toggles as XOR of all inputs.
- **Borrow out** is set when either A < B or borrow propagates through Bin.

**RTL Diagram:**




**Conclusion:**

The **Half Subtractor** and **Full Subtractor** were successfully implemented and simulated in Verilog. The Half Subtractor correctly performed two-bit subtraction, while the Full Subtractor extended it with a borrow input for multi-bit operations. Both circuits produced accurate outputs, confirming the correct working of combinational logic for subtraction.